

Monitorización pasiva de red

Arquitectura, decisiones y operación para sesiones de alto volumen

ESTADO Propuesta para revisión

24 septiembre 2026

CAPTURA → EVENTOS → SESIONES → INVENTARIO
Probe .NET → Kafka + Avro → ClickHouse → PostgreSQL

Criterios que guían el diseño

RED PRIVADA

EJBCA X.509; mTLS en todos los saltos internos.

ESCALA

Millones o miles de millones de sesiones; capacidad por medir.

EQUIPO

Seis personas; .NET y Angular; presupuesto medio.

Enfoque

- Una base operativa y otra analítica, cada una con responsabilidad explícita.
- Workers separados por caudal; backend modular y despliegues escalables.
- Ingestión reintentable, límites de consulta y dimensionamiento con benchmarks.

Contenido

1. Resumen ejecutivo	6. ADRs propuestos
2. Alcance y supuestos	7. Atributos de calidad y objetivos iniciales
3. Diagrama completo de arquitectura lógica	8. Dimensionamiento: datos necesarios
4. Vista de despliegue privado	9. Glosario
5. Flujo fiable de ingestión de sesiones	10. Referencias técnicas

El informe contiene el diagrama lógico, la topología privada, el flujo de ingestión, nueve decisiones de arquitectura, objetivos de calidad, criterios de dimensionamiento y glosario.

1. Resumen ejecutivo

La solución separa tres responsabilidades:

1. **Captura e ingestión:** probes cercanos a las redes monitorizadas publican eventos de sesiones y dispositivos en un clúster privado de Apache Kafka. 2. **Sesiones de alto volumen:** consumidores .NET insertan eventos por lotes en ClickHouse. ClickHouse es el almacenamiento de consulta y análisis del histórico de sesiones. 3.

Inventario y aplicación: PostgreSQL conserva el inventario de dispositivos y los datos administrativos. La API .NET consulta ClickHouse y PostgreSQL; Angular accede solo a la API.

La solución se mantiene modular en el código del backend, pero permite desplegar y escalar por separado la API, el consumidor de sesiones y el consumidor de dispositivos. No se crean microservicios por entidad.

Todos los componentes de producción viven en una red privada. No hay acceso público a Kafka, bases de datos, identidad, esquemas ni paneles. Los operadores acceden desde la red corporativa o mediante VPN. Los probes publican por una interfaz de gestión separada de la interfaz de captura.

La PKI privada se basa en EJBCA y emite certificados X.509 para servicios, nodos y equipos corporativos gestionados. mTLS es obligatorio en todos los saltos de aplicación, datos y operación: también entre el balanceador y cada backend, entre brokers, workers, bases de datos, telemetría y backup. Si un proxy termina TLS, vuelve a establecer mTLS con el siguiente salto; no hay tramos internos en texto plano. El certificado identifica el cliente o equipo; OIDC con Keycloak sigue autenticando a la persona y RBAC autoriza sus acciones.

El diseño evita guardar el mismo histórico de sesiones en PostgreSQL y en un motor de búsqueda adicional. El par PostgreSQL/ClickHouse no es duplicación: PostgreSQL resuelve la gestión transaccional del inventario; ClickHouse, la ingesta y consulta analítica de grandes volúmenes de sesiones.

2. Alcance y supuestos

Requisitos conocidos

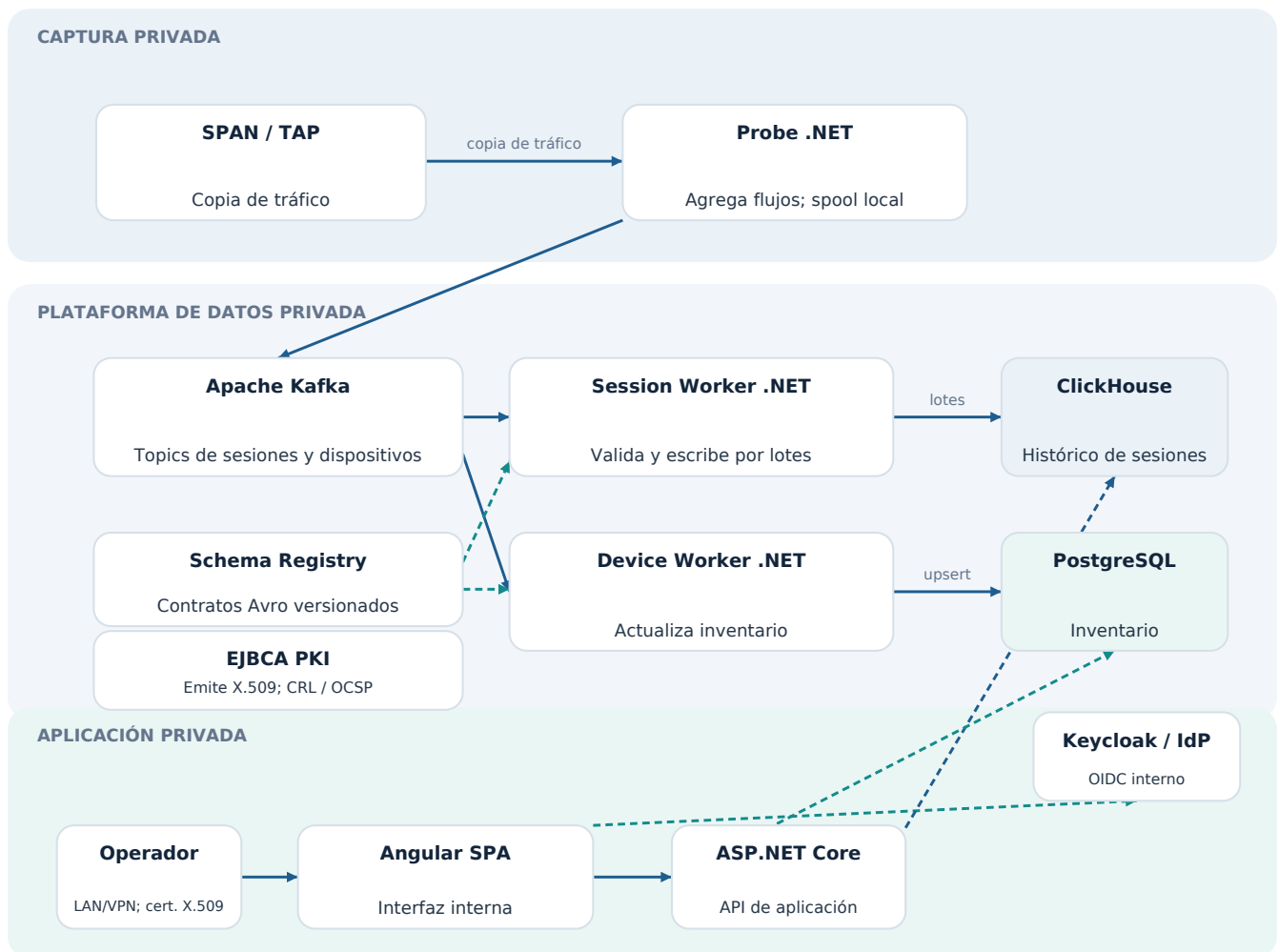
- Monitorización pasiva mediante SPAN o TAP.
- Captura/decodificación con tshark y producción de metadatos de sesiones y dispositivos.
- Consultas de sesiones e inventario desde una aplicación Angular.
- Gestión centralizada de identidad y permisos.
- Comunicaciones internas cifradas y autenticadas.
- PKI privada basada en EJBCA, emisión de certificados X.509 y mTLS obligatorio en todas las comunicaciones internas de la plataforma.
- Volumen de millones o miles de millones de sesiones; se desconoce si es por día, mes o vida del sistema.
- Entorno de red interno y privado.
- Equipo de desarrollo de seis personas, principalmente .NET y Angular; presupuesto medio.

Supuestos que condicionan la propuesta

- El sistema conserva metadatos de sesiones agregadas, no captura ni archiva paquetes PCAP.
- Las sesiones son mayoritariamente append-only: una sesión se inserta y no se actualiza continuamente.
- La búsqueda principal es estructurada: intervalos de tiempo, IP, puerto, protocolo, sitio y probe.
- Los probes pueden guardar temporalmente eventos normalizados en disco si Kafka no está disponible.
- Los usuarios acceden desde equipos corporativos gestionados capaces de presentar un certificado cliente X.509 al portal y a los endpoints internos.
- Se reutiliza la plataforma privada, PKI, balanceador, DNS y sistema de copias de seguridad existentes cuando sea posible.

En esta propuesta, «todas las comunicaciones internas» cubre cada enlace de la aplicación y de los planos de datos y operación. Los protocolos base que no admitan mTLS nativo se mantienen en una red de gestión aislada y usan su mecanismo seguro equivalente; no se degradan a texto plano.

3. Diagrama completo de arquitectura lógica



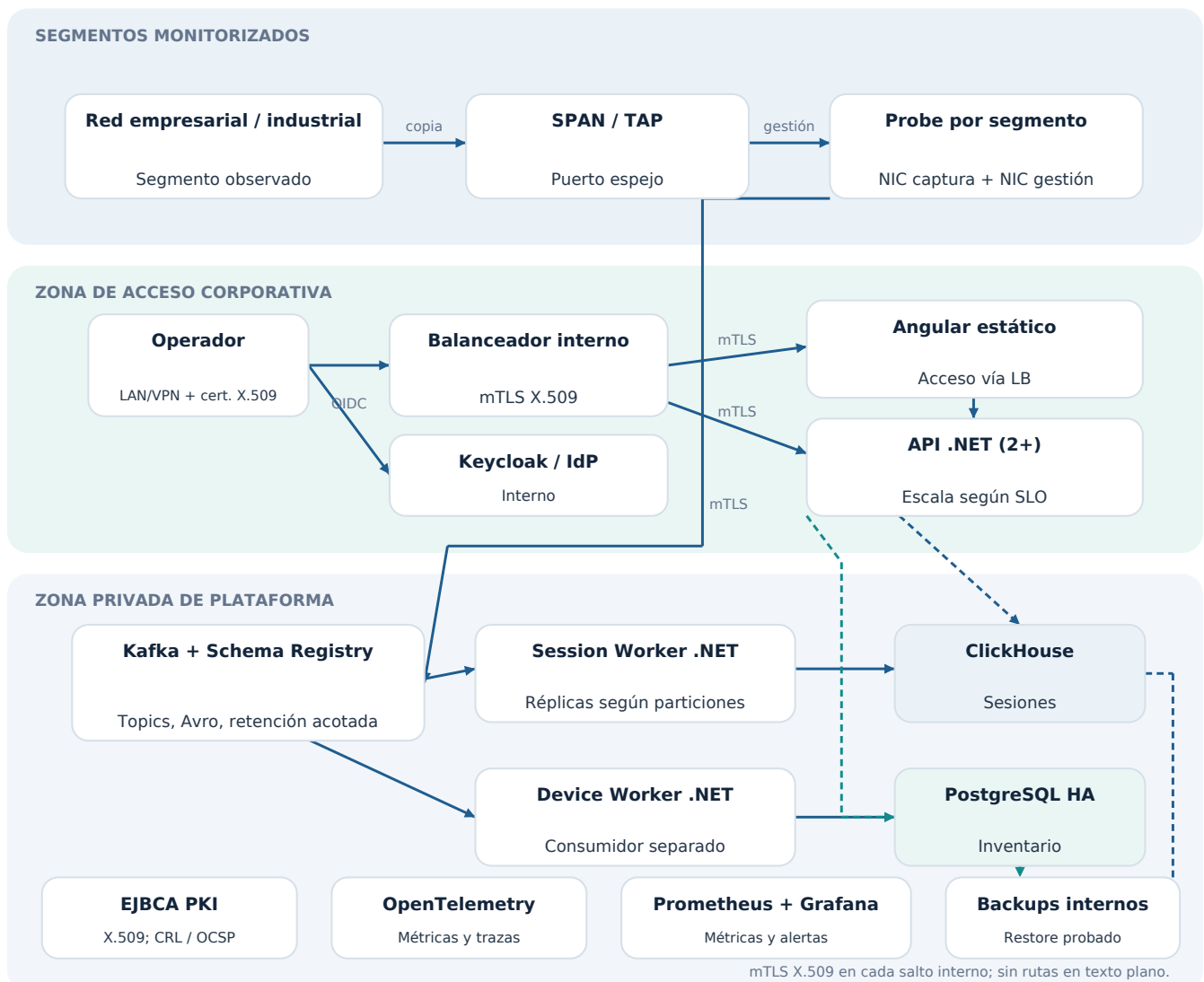
Vista lógica; EJBCA emite las identidades X.509 y mTLS protege todos los canales representados.

Seguridad transversal: todos los enlaces de red representados validan certificado de cliente y servidor mediante mTLS; EJBCA emite y renueva las identidades X.509.

Responsabilidades y límites

- **Probe:** corre en el segmento monitorizado. La interfaz conectada al SPAN/TAP es de captura; la interfaz de gestión inicia conexiones salientes autorizadas hacia Kafka y la plataforma operativa. No recibe tráfico de usuario.
- **Kafka:** desacopla probes y consumidores, absorbe picos y permite reproducir eventos dentro de la ventana de retención. No es el archivo histórico ilimitado.
- **Session Worker:** consume sesiones, valida Avro, forma lotes grandes y escribe en ClickHouse. Tiene permisos para leer el topic de sesiones y escribir únicamente en las tablas de sesiones.
- **Device Worker:** consume observaciones de dispositivos y actualiza el inventario PostgreSQL. No procesa el caudal de sesiones.
- **ClickHouse:** fuente de lectura para sesiones detalladas y agregadas. El modelo se diseña para las consultas reales, no como un JSON genérico.
- **PostgreSQL:** fuente de verdad del inventario editable y datos administrativos.
- **API:** hace de frontera de seguridad y aplicación. La UI nunca se conecta directamente a las bases, Kafka o Schema Registry.
- **Keycloak:** identidad de usuarios humanos. La autenticación de probes y consumidores se resuelve mediante certificados y ACL, no mediante una cuenta de usuario interactiva.

4. Vista de despliegue privado



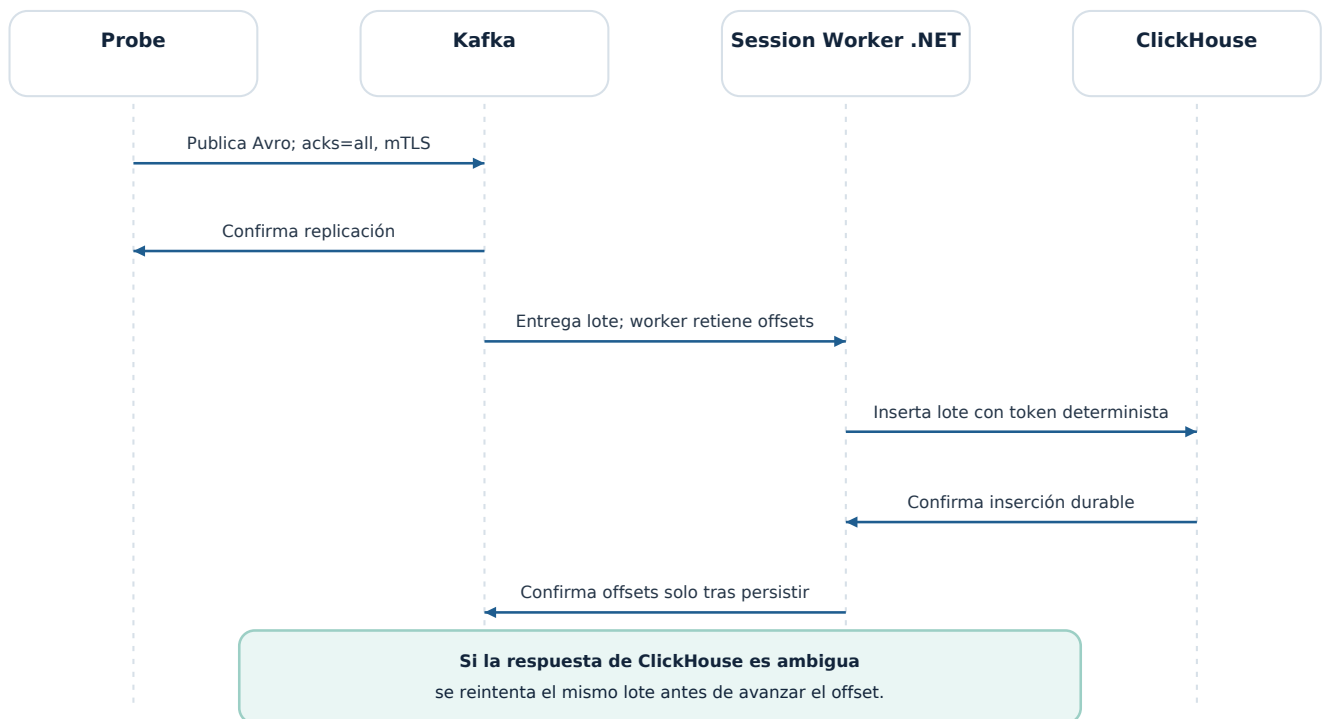
Topología privada: EJBCA, equipos gestionados y mTLS en cada salto, también tras el balanceador.

Regla común: todos los saltos de aplicación, datos y operación usan mTLS X.509, incluido el tramo navegador-balanceador y cada tramo posterior al balanceador. La emisión y renovación depende de EJBCA.

Reglas de red

- Bloquear tráfico entrante desde Internet. El portal acepta solo LAN/VPN corporativa y equipos gestionados con certificado cliente X.509.
- Exigir mTLS cliente-servidor en cada salto de aplicación, datos y operación, incluso tras el balanceador o proxy; sin rutas de aplicación en texto plano.
- Aislar los protocolos de soporte sin mTLS nativo (por ejemplo, DHCP) en una red de gestión con controles propios; no transportan tráfico de aplicación.
- Restringir probes a Kafka, Schema Registry y telemetría; la NIC de captura no enruta. Limitar ClickHouse a API, Session Worker, administración y backup; PostgreSQL a API, Device Worker, administración y backup.
- Aplicar ACL de Kafka por certificado/topic y roles de mínimo privilegio en bases y API.
- Operar EJBCA con raíz offline, CA emisora, perfiles cliente/servidor, CRL/OCSP privados y ciclo X.509 automatizado (emisión, renovación, revocación y alertas); aprovisionar identidad de bootstrap controlada.
- Mantener DNS, hora, PKI, repositorios y copias en la red privada; distribuir parches desde un canal interno.

5. Flujo fiable de ingestión de sesiones



Orden de confirmación que protege frente a reintentos y fallos ambiguos.

La semántica es **at-least-once**, no una promesa de “exactly once” entre dos sistemas independientes. Cada evento lleva un identificador estable. El worker forma lotes repetibles y deriva el token de inserción de topic, partición y rango de offsets. Ante una respuesta ambigua de ClickHouse, reintenta el mismo lote antes de confirmar offsets.

ClickHouse admite tokens de inserción para deduplicar reintentos, pero la ventana de deduplicación es finita. Para reconstrucciones históricas fuera de esa ventana, el procedimiento será cargar una tabla o partición nueva, validar conteos y rangos, y cambiar la lectura tras la comprobación. No se confiará en deduplicación eventual de filas como sustituto de un proceso de recuperación probado.

Si el evento no cumple el esquema, el worker lo envía a un topic de cuarentena/DLQ con causa y metadatos mínimos; solo confirma ese offset después de que la cuarentena sea duradera. No se registra el paquete de red ni datos sensibles sin necesidad.

6. ADRs propuestos

Todos los ADRs tienen estado **Propuesto** hasta que el responsable de producto, seguridad y operación acepte los objetivos y restricciones.

ADR-001 - Backend modular, despliegues por carga

Contexto: seis personas, experiencia .NET/Angular y dominios que comparten reglas de validación e inventario. La API y la ingestión de sesiones tienen perfiles de carga diferentes.

Decisión: mantener un backend modular .NET con módulos de API, sesiones e inventario. Publicar roles ejecutables separados para API, Session Worker y Device Worker; escalar cada rol de forma independiente. El probe es un programa independiente por su proximidad y privilegios de red.

Alternativas consideradas: microservicios independientes por entidad; un único proceso que hospeda UI, captura, API y workers.

Consecuencias: una base de código reduce duplicación de reglas, mientras los despliegues separados permiten escalar ingestión. Los módulos deben tener fronteras claras y no compartir entidades internas como contratos de red.

ADR-002 - Operación íntegra dentro de la red privada

Contexto: el sistema no puede exponer servicios o datos a una red pública. La captura puede conectarse a redes industriales sensibles.

Decisión: desplegar toda la ruta de datos, identidad, esquemas, administración y observabilidad en una zona privada controlada. Acceso de usuario por LAN o VPN desde equipos gestionados. Exigir mTLS con certificados X.509 en todos los saltos de aplicación, datos y operación; cada proxy vuelve a autenticar el siguiente salto y no existe fallback en claro.

Alternativas consideradas: servicios SaaS públicos; componentes mixtos entre nube pública y red local.

Consecuencias: mayor control de datos y conectividad, a cambio de que el equipo interno gestione disponibilidad, actualización, DNS, hora, certificados y copias. La renovación automática y las alertas de vencimiento son requisitos operativos, no tareas manuales periódicas. Debe existir un mecanismo de parcheo y distribución de artefactos que funcione sin acceso público directo.

ADR-003 - Kafka como backbone de eventos

Contexto: varios probes producen eventos a alta velocidad; los consumidores deben poder absorber picos, recuperarse y evolucionar independientemente.

Decisión: un solo clúster Kafka privado con topics separados de sesiones y dispositivos. Replicación y particiones se dimensionan con pruebas de caudal, retención y paralelismo. Como punto de partida de alta disponibilidad: factor de replicación 3, min.insync.replicas 2, productor acks=all e idempotencia del productor habilitada.

Alternativas consideradas: HTTP directo del probe al backend; introducir dos brokers distintos.

Consecuencias: backpressure, buffer y replay durante una ventana acotada. Kafka no reemplaza al histórico en ClickHouse ni al inventario en PostgreSQL. Tres brokers son un punto de partida de HA, no una estimación de capacidad. Las particiones deben permitir suficiente paralelismo sin perder el orden requerido por cada clave.

ADR-004 - Avro y un Schema Registry interno

Contexto: el volumen hace relevante el tamaño de los eventos, y probe y consumidores pueden evolucionar en despliegues distintos.

Decisión: usar Avro binario para eventos y un solo Schema Registry desplegado internamente. Aplicar una política de compatibilidad hacia atrás para cambios compatibles; un cambio incompatible requiere una transición explícita de versión/topic. Validar esa política en CI.

Alternativas consideradas: JSON sin registro central; DLL compartida con tipos .NET; formatos binarios múltiples.

Consecuencias: menor sobrecarga de payload y contratos verificables, a cambio de operar y respaldar el Registry. No compartir DLLs entre probe, API y workers. Avro está diseñado como un formato binario compacto con esquemas asociados. [1]

ADR-005 - ClickHouse para sesiones, PostgreSQL para inventario

Contexto: el histórico de sesiones puede alcanzar miles de millones de filas; el inventario requiere altas, edición, unicidad y transacciones.

Decisión: ClickHouse almacena las sesiones y agregados de consulta. PostgreSQL almacena dispositivos, configuración y datos administrativos. No duplicar el histórico completo de sesiones en PostgreSQL ni añadir Elasticsearch en la primera versión.

Alternativas consideradas: PostgreSQL como base de todas las sesiones; Elasticsearch como segunda copia analítica; TimescaleDB más PostgreSQL.

Consecuencias: cada base tiene una función acotada. La API implementa consultas mediante dos adaptadores de persistencia. ClickHouse puede ejecutarse on-premises y distribuirse; la topología concreta se determina por benchmark, almacenamiento y disponibilidad. [2]

ADR-006 - Consumidor .NET explícito y escrituras por lotes

Contexto: el sistema debe controlar la relación entre offsets Kafka, datos persistidos y reintentos.

Decisión: el Session Worker consume Kafka, deserializa Avro, valida, forma lotes y escribe en ClickHouse antes de confirmar offsets. El Device Worker procesa el topic de dispositivos por separado y actualiza PostgreSQL. Cada flujo cuenta con DLQ, reintentos con backoff y métricas propias.

Alternativas consideradas: consumo integrado de Kafka dentro de ClickHouse; Kafka Connect con un sink externo.

Consecuencias: control preciso del flujo de confirmación y uso del stack .NET conocido. Se debe probar la carga de lotes y la recuperación ante fallos ambiguos. El motor Kafka de ClickHouse requiere configuración explícita de durabilidad para evitar ventanas donde el offset ya esté confirmado antes de que los datos estén sincronizados; por eso no se usa con valores por defecto. [5]

ADR-007 - Diseño físico y retención de ClickHouse

Contexto: la retención y el patrón de consulta determinan coste y latencia más que el número total de sesiones por sí solo.

Decisión: tablas con columnas explícitas y tipadas; datos temporales en UTC; particiones por tiempo para ciclo de vida y TTL; ordenación de tabla elegida a partir de filtros reales, como sitio, hora e IP. No particionar por IP, dispositivo ni identificadores de cardinalidad alta. Crear rollups/materialized views solo para consultas recurrentes demostradas.

Alternativas consideradas: tabla genérica con todo el evento como JSON; particiones por cada sitio, dispositivo o día sin medir; conservar detalle indefinidamente.

Consecuencias: las consultas comunes pueden saltar rangos de datos y la retención puede eliminar particiones completas. El orden de MergeTree influye directamente en selección de rangos; la partición sirve sobre todo para gestión de datos y retención, no debe asumirse que toda partición acelera consultas. [3, 4]

ADR-008 - PKI EJBCA y mTLS en todos los saltos internos

Contexto: todos los enlaces internos de la aplicación y de los planos de datos y operación deben cifrar y autenticar ambos extremos. Operadores, servicios, brokers, bases de datos y proxies requieren identidades distintas. Una conexión TLS solo autentica al servidor y no satisface el requisito.

Decisión: EJBCA es la PKI privada de la solución y emite certificados X.509 con perfiles cliente/servidor para probes, proxies, API, Keycloak, Kafka, Schema Registry, workers, ClickHouse, PostgreSQL, observabilidad, backup y equipos corporativos gestionados. mTLS es obligatorio en cada salto de aplicación, datos y operación: cliente y servidor validan cadena, SAN, uso extendido del certificado y vigencia. El balanceador vuelve a establecer mTLS hacia cada backend; se prohíbe el downgrade a texto plano. La CA raíz se mantiene fuera de línea y una CA emisora interna firma certificados operativos. EJBCA publica CRL/OCSP en la red privada; emisión, renovación, revocación y alertas de expiración se automatizan. El bootstrap usa una identidad de aprovisionamiento controlada. Keycloak conserva OIDC Authorization Code con PKCE para identificar a la persona; el certificado identifica al equipo, no sustituye el login ni el RBAC. Kafka ACL, roles de bases de datos y autorización de API mantienen mínimo privilegio. EJBCA permite definir perfiles por uso del certificado y publicar su estado mediante CRL/OCSP. [10]

Alternativas consideradas: TLS con autenticación solo de servidor; una CA distinta por componente; reutilizar una PKI corporativa sin controlar perfiles, renovación ni identidades de servicio.

Consecuencias: autenticación mutua coherente entre todos los saltos y atribución de conexiones a identidades concretas. Hay que probar mTLS en cada protocolo y cliente (incluidas conexiones entre brokers y nodos), gestionar almacenes de confianza, rotación y renovación sin caída, revocaciones, respaldo y alta disponibilidad de EJBCA. Los navegadores requieren certificados corporativos distribuidos a los dispositivos gestionados. OIDC/RBAC sigue siendo necesario porque mTLS no autoriza acciones de usuario. Kafka documenta autenticación de broker/cliente; ClickHouse, PostgreSQL y Keycloak documentan configuración de TLS y certificados cliente. [11, 12, 13, 14]

ADR-009 - Despliegue privado, backup y observabilidad

Contexto: hay que operar una plataforma de datos de alto volumen en red privada con un equipo de desarrollo de seis personas.

Decisión: usar VMs Linux o una plataforma privada de contenedores existente. No introducir Kubernetes solo para este proyecto. Utilizar el mecanismo corporativo de backups, restauraciones, DNS y repositorio interno de imágenes. Instrumentar con OpenTelemetry y consolidar métricas y alertas en Prometheus/Grafana internos o la plataforma corporativa existente.

Alternativas consideradas: nueva plataforma Kubernetes; monitorización separada por componente; backups alojados en servicios públicos.

Consecuencias: menos inversión inicial en plataforma, pero siguen haciendo falta responsables de operación de Kafka, ClickHouse, PostgreSQL, EJBCA, identidad y certificados. La restauración y la rotación/renovación de certificados deben ensayarse, no solo configurarse.

7. Atributos de calidad y objetivos iniciales

Los valores siguientes son **objetivos de partida para validar**, no garantías sin pruebas de carga y una infraestructura dimensionada.

Atributo	Medida y escenario propuesto
Integridad de ingestión	Ningún evento cuya escritura haya sido confirmada por Kafka debe perderse. Medir eventos de origen, persistidos, puestos en cuarentena y descartados por probe.
Frescura	p95 menor de 10 segundos desde la publicación confirmada en Kafka hasta que la sesión sea consultable en ClickHouse, bajo la carga acordada.
Rendimiento de consulta	p95 menor de 2 segundos para consultas interactivas definidas, con intervalo temporal y límite de resultados acordados. Las consultas históricas amplias deben pasar a exportación asíncrona o agregados.

Atributo	Medida y escenario propuesto
Capacidad	Prueba sostenida a 2x la tasa pico proyectada; medir throughput, lag, latencia, uso de disco, memoria, merges y captura perdida.
Recuperación	El consumidor debe reanudar desde offsets confirmados. Definir con negocio RPO/RTO; no suponer que réplica equivale a backup.
Seguridad	Cero endpoints públicos. mTLS X.509 obligatorio en todos los saltos de aplicación, datos y operación, incluidos equipos gestionados y saltos tras proxy; sin texto plano. Medir fallos de cadena/SAN/EKU, accesos denegados y cambios de ACL/roles.
Operabilidad	Alertar por caída de probes, disco del spool, lag y réplicas, partes/merges de ClickHouse, fallos de backup y certificados próximos a vencer, renovaciones fallidas o revocaciones.
Mantenibilidad	Cambios de esquema incompatibles bloquean CI. Solo API/workers tienen credenciales de escritura; probe no tiene acceso a las bases.

Las consultas interactivas deben imponer límites: rango de tiempo, tamaño de página, columnas seleccionadas y tiempo máximo de ejecución. Una búsqueda de varios meses o una exportación de millones de sesiones no debe competir sin límite con las consultas de los operadores.

8. Dimensionamiento: datos necesarios

No fijar número de nodos ni almacenamiento sin:

- tasa media y pico de sesiones por segundo;
- número de probes, sitios y enlaces monitorizados;
- tamaño medio del evento Avro tras agregación;
- retención del detalle, de agregados y de Kafka;
- consultas más frecuentes, filtros combinados y concurrencia de usuarios;
- tiempo máximo de recuperación tras caída de un broker o nodo de ClickHouse;
- RPO/RTO y disponibilidad requeridos;
- tolerancia a pérdida de captura medida en cada NIC/probe.

Fórmulas de estimación inicial:

- Filas/día = eventos por segundo x 86.400.
- Volumen de Kafka $\approx$ bytes/evento x eventos/segundo x segundos de retención x factor de replicación, ajustado por compresión y overhead medidos.
- Volumen de ClickHouse $\approx$ filas retenidas x bytes comprimidos por fila, multiplicado por réplicas y ajustado por partes, índices, agregados y margen operativo.

El número total de filas no basta para dimensionar: hacen falta tasa media y pico, tamaño serializado, retención y consultas. Como referencia, mil millones de eventos al día equivalen a unos 11.600 eventos por segundo de media; los picos pueden ser bastante mayores.

El tamaño comprimido se mide con un dataset representativo: la cardinalidad de IPs, campos, índices y ordenación altera mucho el resultado.

9. Glosario

Término	Qué es y qué hace en esta arquitectura
Probe	Agente cercano al segmento de red; captura metadatos, agrega flujos y envía eventos.
SPAN / TAP	Mecanismo de red que entrega al sensor una copia del tráfico sin colocarlo en el camino de producción.
tshark	Herramienta CLI de Wireshark para capturar y decodificar protocolos.

Término	Qué es y qué hace en esta arquitectura
Apache Kafka	Plataforma de eventos que conserva temporalmente mensajes, desacopla productores y consumidores y permite replay dentro de su retención.
Topic / partición / offset	Topic agrupa eventos por tipo; particiones permiten paralelismo; offset identifica la posición de un consumidor en una partición.
Apache Avro	Formato de eventos binario y compacto que se interpreta usando un esquema.
Schema Registry	Servicio interno que versiona esquemas y permite controlar compatibilidad de eventos.
ClickHouse	Base columnar distribuida para ingestión y consulta analítica de sesiones de gran volumen.
MergeTree	Familia de tablas de ClickHouse con partes ordenadas, índices dispersos, merges y opciones de TTL/replicación.
ClickHouse Keeper	Servicio de coordinación utilizado por instalaciones autogestionadas de ClickHouse para replicación y metadatos de clúster.
PostgreSQL	Base relacional transaccional para el inventario de dispositivos y administración.
ASP.NET Core API	API interna que valida acceso y aplica reglas de negocio para UI, consultas y gestión.
.NET Worker	Proceso de background que consume Kafka, agrupa eventos y escribe en el almacén destino.
Angular SPA	Interfaz web interna, servida como recursos estáticos y conectada exclusivamente a la API.
Keycloak	Proveedor de identidad y autorización OIDC autogestionado dentro de la red privada.
OIDC	Capa de identidad sobre OAuth 2.0 usada para iniciar sesión y emitir tokens.
RBAC	Control de acceso basado en roles; la API decide qué puede hacer cada usuario.
PKI / CA	Infraestructura y autoridad que emiten, renuevan y revocan certificados.
EJBCA	PKI y autoridad certificadora privada de la solución; emite y gestiona certificados X.509 para servicios, nodos y equipos.
X.509	Formato de certificado que vincula la identidad de un servicio o equipo con su clave pública y sus usos permitidos.
SAN / EKU	SAN identifica nombres o identidades del certificado; EKU limita su uso, por ejemplo a autenticación de cliente o servidor.
TLS / mTLS	TLS cifra y autentica al servidor; mTLS exige y valida certificados tanto de cliente como de servidor en cada conexión.
CRL / OCSP	Mecanismos para consultar o distribuir el estado de revocación de certificados.
DLQ / cuarentena	Topic separado para eventos que no se pueden procesar, con causa y metadatos mínimos.
OpenTelemetry	API y formato neutral para emitir métricas y trazas desde aplicaciones.
Prometheus / Grafana	Prometheus recoge métricas; Grafana ofrece paneles y alertas dentro de la red privada.
Contenedor	Empaqueta una aplicación con sus dependencias; no obliga a usar Kubernetes.

Exclusiones y alternativas

No se incorporan Elasticsearch, Kafka Connect, DLL de dominio compartida, Integration Console, TimescaleDB ni Kubernetes. Sus alternativas y motivos se recogen en los ADR-001, ADR-004, ADR-005, ADR-006 y ADR-009.

10. Referencias técnicas

[Apache Avro - documentación oficial](#)[Keycloak - OpenID Connect](#)[ClickHouse - despliegue on-premises y modos de operación](#)[OpenTelemetry - documentación](#)[ClickHouse - MergeTree, ordenación, claves y particiones](#)[EJBCA - conceptos de CA, perfiles y entidades](#)[ClickHouse - TTL y ciclo de vida de datos](#)[Apache Kafka - cifrado y autenticación SSL](#)[ClickHouse - integración con Kafka y materialized views](#)[ClickHouse - configuración TLS](#)[ClickHouse - inserciones idempotentes y deduplicación](#)[PostgreSQL - TLS y certificados cliente](#)[Apache Kafka - configuración de acks y réplicas sincronizadas](#)[Keycloak - configuración mTLS](#)